

Chapter 16. Multithreading

Multithreading enables an application to execute several pieces of code simultaneously. There are two common reasons for doing this. One is to exploit the computer's parallel processing capabilities—multicore CPUs are now more or less ubiquitous, and to realize their full performance potential, you'll need to provide the CPU with multiple streams of work to give all of the cores something useful to do. The other usual reason for writing multithreaded code is to prevent progress from grinding to a halt when you do something slow, such as reading from disk.

Multithreading is not the only way to solve that second problem—asynchronous techniques can be preferable. C# has features for supporting asynchronous work. Asynchronous execution doesn't necessarily mean multithreading, but the two are often related in practice, and I will be describing some of the asynchronous programming models in this chapter. However, this chapter focuses on the threading foundations. I will describe the language-level support for asynchronous code in [Chapter 17](#).

Threads

All the operating systems that .NET can run on allow each process to contain multiple threads (although if you build to Web Assembly and run code in the browser, that particular environment currently doesn't support creation of new threads). Each thread has its own stack, and the OS presents the illusion that a thread gets a whole CPU *hardware thread* to itself. (See the sidebar, [“Processors, Cores, and Hardware Threads.”](#)) You can create far more OS threads than the number of hardware threads your computer provides, because the OS virtualizes the CPU, context switching from one thread to another. The computer I'm using as I write this has 16 hardware threads, which is a reasonably generous quantity

but some way short of the 8,893 threads currently active across the various processes running on my machine.

A *hardware thread* is one piece of hardware capable of executing code. Back in the early 2000s, one processor chip gave you one hardware thread, and you got multiple hardware threads only in computers that had multiple, physically separate CPUs plugged into separate sockets on the motherboard. However, two inventions have made the relationship between hardware and threads more complex: multicore CPUs and hyperthreading.

With a multicore CPU, you effectively get multiple processors on a single piece of silicon. This means that opening up your computer and counting the number of processor chips doesn't necessarily tell you how many hardware threads you've got. But if you were to inspect the CPU's silicon with a suitable microscope, you'd see two or more distinct processors next to each other on the chip.

Hyperthreading, also known as simultaneous multithreading (SMT), complicates matters further. A hyperthreaded core is a single processor that has two sets of certain parts. (It could be more than two, but doubling seems most common.) So, although there might be only a single part of the core capable of performing, say, floating-point division, there will be two sets of registers. Each set of registers includes an instruction pointer (IP) register that keeps track of where execution has reached. Registers also contain the immediate working state of the code, so by having two sets, a single core can run code from two places at once—in other words, hyperthreading enables a single core to provide two hardware threads. Since only certain parts of the CPU are doubled up, two execution contexts have to share some resources—they can't both perform floating-point division operations simultaneously, because there's only one piece of hardware in the core to do that. However, if one of the hardware threads wants to do some division while another multiplies two numbers together, they will typically be able to do so in parallel, because those operations are performed by different areas of the core. Hyperthreading enables more parts of a single CPU core to be kept busy simultaneously. It doesn't give you quite the same throughput as two full cores (because if the two hardware threads both want to do the same kind of work at once,

one of them will have to wait), but it can often provide better throughput from each core than would otherwise be possible.

In a hyperthreaded system, the total number of hardware threads available is the number of cores multiplied by the number of hyperthreaded execution units per core. For example, the Intel Core i9-9900K processor has eight cores with two-way hyperthreading, giving a total of 16 hardware threads.

The CLR presents its own threading abstraction on top of OS threads. In .NET, there is always a direct relationship—each `Thread` object corresponds directly to some particular underlying OS thread. On .NET Framework, this relationship is not guaranteed to exist—applications that use the CLR’s unmanaged hosting API to customize the relationship between the CLR and its containing process can in theory cause a CLR thread to move between different OS threads. In practice, this capability is very rarely used, so even on .NET Framework, each CLR thread will usually correspond to one OS thread.

I will get to the `Thread` class shortly, but before writing multithreaded code, you need to understand the ground rules for managing state¹ when using multiple threads.

Threads, Variables, and Shared State

Each CLR thread gets various thread-specific resources, such as the call stack (which holds method arguments and some local variables). Because each thread has its own stack, the local variables that end up there will be local to the thread. Each time you invoke a method, you get a new set of its local variables. Recursion relies on this, but it’s also important in multithreaded code, because data that is accessible to multiple threads requires much more care, particularly if that data changes. Coordinating access to shared data is complex. I’ll be describing some of the techniques for that in the section [“Synchronization”](#), but it’s better to avoid the problem entirely where possible, and the thread-local nature of the stack can be a great help.

For example, consider a web-based application. Busy sites have to handle requests from multiple users simultaneously. ASP.NET Core uses multithreading to support this, so you're likely to end up in a situation where a particular piece of code (e.g., the code for your site's home page) is being executed simultaneously on several different threads. (Websites typically don't just serve up the exact same content every time, because pages are often tailored to particular users, so if 1,000 users ask to see the home page, it will run the code that generates that page 1,000 times.) ASP.NET Core provides you with various objects that your code will need to use, but most of these are specific to a particular request. So, if your code is able to work entirely with those objects and with local variables, each thread can operate completely independently. If you need shared state (such as objects that are visible to multiple threads, perhaps through a static field or property), life will get more difficult, but local variables are usually straightforward.

Why only “usually”? Things get more complex if you use lambdas or anonymous functions, because they make it possible to declare a variable in a containing method and then use that in an inner method. This variable is now available to two or more methods, and with multithreading, it's possible that these methods could execute concurrently. (As far as the CLR is concerned, it's not really a local variable anymore—it's a field in a compiler-generated class.) Sharing local variables across multiple methods removes the guarantee of complete locality, so you need to take the same sort of care with such variables as you would with more obviously shared items, like static properties and fields.

Another important point to remember in multithreaded environments is the distinction between a variable and the object it refers to. (This is an issue only with reference type variables.) Although a local variable is accessible only inside its declaring method, that variable may not be the only one that refers to a particular object. Sometimes it will be—if you create the object inside the method and never store it anywhere that would make it accessible to a wider audience, then you have nothing to worry about. The `StringBuilder` that [Example 16-1](#) creates is only ever used within the method that creates it.

Example 16-1. Object visibility and methods

```
public static string FormatDictionary<TKey, TValue>(
    IDictionary<TKey, TValue> input)
{
    var sb = new StringBuilder();
    foreach ((TKey key, TValue value) in input)
    {
        sb.Append($"{key}: {value}");
        sb.AppendLine();
    }

    return sb.ToString();
}
```

This code does not need to worry about whether other threads might be trying to modify the `StringBuilder`. There are no nested methods here, so the `sb` variable is truly local, and that's the only thing that contains a reference to the `StringBuilder`. (This relies on the fact that the `StringBuilder` doesn't sneakily store copies of its `this` reference anywhere that other threads might be able to see.)

But what about the `input` argument? That's also local to the method, but the object it refers to is not: the code that calls `FormatDictionary` gets to decide what `input` refers to. Looking at [Example 16-1](#) in isolation, it's not possible to say whether the dictionary object to which it refers is currently in use by other threads. The calling code could create a single dictionary and then create two threads, and have one modify the dictionary while the other calls this `FormatDictionary` method. This would cause a problem: most dictionary implementations do not support being modified on one thread at the same time as being used on some other thread. And even if you were working with a collection that was designed to cope with concurrent use, you're often not allowed to modify a collection while an enumeration of its contents is in progress (e.g., a `foreach` loop).

You might think that any collection designed to be used from multiple threads simultaneously (a *thread-safe* collection, you might say) should allow one thread to iterate over its contents while another modifies the

contents. If it disallows this, then in what sense is it thread safe? In fact, the main difference between a thread-safe and a non-thread-safe collection in this scenario is predictability: whereas a thread-safe collection might throw an exception when it detects that this has happened, a non-thread-safe collection does not guarantee to do anything in particular. It might crash, or you might start getting perplexing results from the iteration, such as a single entry appearing multiple times. It could do more or less anything because you're using it in an unsupported way. Sometimes, thread safety just means that failure happens in a well-defined and predictable manner.

As it happens, the various collections in the `System.Collections.Concurrent` namespace do in fact support changes while enumeration is in progress without throwing exceptions. However, they often have a different API from the other collection classes specifically to support concurrency, so they are not always drop-in replacements.

There's nothing [Example 16-1](#) can do to ensure that it uses its `input` argument safely in multithreaded environments, because it is at the mercy of its callers. Concurrency hazards need to be dealt with at a higher level. In fact, the term *thread safe* is potentially misleading, because it suggests something that is not, in general, possible. Inexperienced developers often fall into the trap of thinking that they are absolved of all responsibility for thinking about threading issues in their code by just making sure that all the objects they're using are thread safe. This usually doesn't work, because while individual thread-safe objects will maintain their own integrity, that's no guarantee that your application's state as a whole will be coherent.

To illustrate this, [Example 16-2](#) uses the `ConcurrentDictionary<TKey, TValue>` class from the `System.Collections.Concurrent` namespace. Every operation this class defines is thread safe in the sense that each will leave the object in a consistent state and will produce the expected result given the collection's state prior to the call. However, this example contrives to use it in a non-thread-safe fashion.

Example 16-2. Non-thread-safe use of a thread-safe collection

```
static string UseDictionary(ConcurrentDictionary<int, string> cd)
{
    cd[1] = "One";
    return cd[1];
}
```

This seems like it could not fail. (It also seems pointless; that's just to show how even a very simple piece of code can go wrong.) But if the dictionary instance is being used by multiple threads (which seems likely, given that we've chosen a type designed specifically for multithreaded use), it's entirely possible that in between setting a value for key 1 and trying to retrieve it, some other thread will have removed that entry. If I put this code into a program that repeatedly runs this method on several threads, but that also has several other threads busily removing the very same entry, I eventually see a `KeyNotFoundException`.

Concurrent systems need a top-down strategy to ensure system-wide consistency. (This is why database management systems often group sets of operations together as transactions, atomic units of work that either succeed completely or have no effect at all.) Looking at [Example 16-1](#), this means that it is the responsibility of code that calls `FormatDictionary` to ensure that the dictionary can be used freely for the duration of the method.

WARNING

Although calling code should guarantee that whatever objects it passes are safe to use for the duration of a method call, you cannot in general assume that it's OK to hold on to references to your arguments for future use. Anonymous functions and delegates make it easy to do this accidentally—if a nested method refers to its containing method's arguments, and if that nested method runs after the containing method returns, it may no longer be safe to assume that you're allowed to access the objects to which the arguments refer. If you need to do this, you will need to document the assumptions you're making about when you can use objects, and inspect any code that calls the method to make sure that these assumptions are valid.

Thread-Local Storage

Sometimes it can be useful to maintain thread-local state at a broader scope than a single method. Various parts of the runtime libraries do this. For example, the `System.Transactions` namespace defines an API for using transactions with databases, message queues, and any other resource managers that support them. It provides an implicit model where you can start an *ambient transaction*, and any operations that support this will enlist in it without you needing to pass any explicit transaction-related arguments. (It also supports an explicit model, should you prefer that.) The `Transaction` class's static `Current` property returns the ambient transaction for the current thread, or it returns `null` if the thread currently has no ambient transaction in progress.

To support this sort of per-thread state, .NET offers the `ThreadLocal<T>` class. [Example 16-3](#) uses this to provide a wrapper around a delegate that allows only a single call into the delegate to be in progress on any one thread at any time.

Example 16-3. Using `ThreadLocal<T>`

```
public class Notifier(Action callback)
{
    private readonly ThreadLocal<bool> _isCallbackInProgress = new();

    public void Notify()
    {
        if (_isCallbackInProgress.Value)
        {
            throw new InvalidOperationException(
                "Notification already in progress on this thread");
        }

        try
        {
            _isCallbackInProgress.Value = true;
            callback();
        }
        finally
        {

```

```
        _isCallbackInProgress.Value = false;
    }
}
}
```

If the method that `Notify` calls back attempts to make another call to `Notify`, this will block that attempt at recursion by throwing an exception. However, because it uses a `ThreadLocal<bool>` to track whether a call is in progress, this will allow simultaneous calls as long as each call happens on a separate thread.

You get and set the value that `ThreadLocal<T>` holds for the current thread through the `Value` property. The constructor is overloaded, and you can pass a `Func<T>` that will be called back each time a new thread first tries to retrieve the value to create a default initial value. (The initialization is lazy—the callback won't run every time a new thread starts. A `ThreadLocal<T>` invokes the callback only the first time a thread attempts to use the value.) There is no fixed limit to the number of `ThreadLocal<T>` objects you can create.

There's one thing you need to be careful about with thread-local storage. If you create a new object for each thread, be aware that an application might create a large number of threads over its lifetime, especially if you use the thread pool (which is described in detail later). If the per-thread objects you create are expensive, this might cause problems. Furthermore, if there are any disposable per-thread resources, you will not necessarily know when a thread terminates; the thread pool regularly creates and destroys threads without telling you when it does so.

If you don't need the automatic creation each time a new thread first uses thread-local storage, you can instead just annotate a static field with the `[ThreadStatic]` attribute. This is handled by the CLR: it effectively means that each thread that accesses this field gets its own distinct field. This can reduce the number of objects that need to be allocated. But be careful: it's possible to define a field initializer for such fields, but that initializer will run only for the first thread to access the field. For other threads using the same `[ThreadStatic]`, the field will initially contain the default zero-like value for the field's type.

One last note of caution: be wary of thread-local storage (and any mechanism based on it) if you plan to use the asynchronous language features described in [Chapter 17](#), because those make it possible for a single invocation of a method to use multiple different threads as it progresses. This would make it a bad idea for that sort of method to use ambient transactions, or anything else that relies on thread-local state. Many .NET features that you might think would use thread-local storage (e.g., the ASP.NET Core framework's static `HttpContext.Current` property, which returns an object relating to the HTTP request that the current thread is handling) turn out to associate information with something called the *execution context* instead. An execution context is more flexible, because it can hop across threads when required. I'll be describing it later.

For the issues I've just discussed to be relevant, we'll need to have multiple threads. There are four main ways to use multithreading. In one, the code runs in a framework that creates multiple threads on your behalf, such as ASP.NET Core. Another is to use certain kinds of callback-based APIs. A few common patterns for this are described in [“Tasks”](#) and [“Other Asynchronous Patterns”](#). But the two most direct ways to use threads are to create new threads explicitly or to use the .NET thread pool.

The Thread Class

As I mentioned earlier, the `Thread` class (defined in the `System.Threading` namespace) represents a CLR thread. You can obtain a reference to the `Thread` object representing the thread that's executing your code with the `Thread.CurrentThread` property, but if you're looking to introduce some multithreading, you can construct a new `Thread` object.

A new thread needs to know what code it should run when it starts, so you must provide a delegate, and the thread will invoke the method the delegate refers to when it starts. The thread will run until that method returns normally, or allows an exception to propagate all the way to the top of the stack (or the thread is forcibly terminated through any of the OS mechanisms for killing threads or their containing processes).

[Example 16-4](#) creates three threads to download the contents of three

web pages simultaneously. It uses a single `HttpClient` instance to do this, which is OK because that type is designed to be used concurrently from multiple threads.

Example 16-4. Creating threads

```
internal static class Program
{
    private static readonly HttpClient http = new();

    private static void Main()
    {
        Thread t1 = new(MyThreadEntryPoint);
        Thread t2 = new(MyThreadEntryPoint);
        Thread t3 = new(MyThreadEntryPoint);

        t1.Start("https://endjin.com/");
        t2.Start("https://oreilly-com.ezproxy.sfp1.org/");
        t3.Start("https://dotnet.microsoft.com/");
    }

    private static void MyThreadEntryPoint(object? arg)
    {
        string url = (string)arg!;

        Console.WriteLine($"Downloading {url}");
        var response = http.Send(new HttpRequestMessage(HttpMethod.Get, url));
        using StreamReader r = new(response.Content.ReadAsStream());
        string page = r.ReadToEnd();
        Console.WriteLine($"Downloaded {url}, length {page.Length}");
    }
}
```

The `Thread` constructor is overloaded and accepts two delegate types. The `ThreadStart` delegate requires a method that takes no arguments and returns no value, but in [Example 16-4](#), the `MyThreadEntryPoint` method takes a single `object` argument, which matches the other delegate type, `ParameterizedThreadStart`. This provides a way to pass an argument to each thread, which is useful if you're invoking the same method on several different threads, as this example does. The thread

will not run until you call `Start`, and if you're using the `ParameterizedThreadStart` delegate type, you must call the overload that takes a single `object` argument. I'm using this to make each thread download from a different URL.

There are two more overloads of the `Thread` constructor, each adding an `int` argument after the delegate argument. This `int` specifies the size of stack for the thread. Current .NET implementations require stacks to be contiguous in memory, making it necessary to preallocate address space for the stack. If a thread exhausts this space, the CLR throws a `StackOverflowException`. (You normally see those only when a bug causes infinite recursion.) Without this argument, the CLR will use the default stack size for the process. (This varies by OS; on Windows it will usually be 1 MB.) You can change it by setting the `DOTNET_DefaultStackSize` environment variable. Note that it interprets the value as a hexadecimal number. It's rare to need to change this but not unheard of. If you have recursive code that produces very deep stacks, you might need to run it on a thread with a larger stack. Conversely, if you're creating huge numbers of threads, you might want to reduce the stack size to conserve resources, because the default of 1 MB is usually considerably more than is really required. However, it's usually not a great idea to create such a large number of threads. So, in most cases, you will create only a moderate number of threads and just use the constructors that use the default stack size.

Notice that the `Main` method in [Example 16-4](#) returns immediately after starting the three threads. Despite this, the application continues to run—it will run until all the threads finish. The CLR keeps the process alive until there are no *foreground threads* running, where a foreground thread is defined to be any thread that hasn't explicitly been designated as a background thread. If you want to prevent a particular thread from keeping the process running, set its `IsBackground` property to `true`. (This means that background threads may be terminated while they're in the middle of doing something, so you need to be careful about what kind of work you do on these threads.)

Creating threads directly is not the only option. The thread pool provides a commonly used alternative.

The Thread Pool

On most operating systems, it is relatively expensive to create and shut down threads. If you need to perform a fairly short piece of work (such as serving up a web page or some similarly brief operation), it would be a bad idea to create a thread just for that job and to shut it down when the work completes. There are two serious problems with this strategy: first, you may end up expending more resources on the startup and shutdown costs than on useful work; second, if you keep creating new threads as more work comes in, the system may bog down under load—with heavy workloads, creating ever more threads will tend to reduce throughput. This is because, in addition to basic per-thread overheads such as the memory required for the stack, the OS needs to switch regularly between runnable threads to enable them all to make progress, and this switching has its own overheads.

To avoid these problems, .NET provides a thread pool. You can supply a delegate that the runtime will invoke on a thread from the pool. If necessary, it will create a new thread, but where possible, it will reuse one it created earlier, and it might make your work wait in a queue if all the threads created so far are busy. After your method runs, the CLR will not normally terminate the thread; instead, the thread will stay in the pool waiting for other work items, which amortizes the cost of creating the thread over multiple work items. It will create new threads if necessary, but it tries to keep the thread count at a level that results in the number of runnable threads matching the hardware thread count, to minimize switching costs.

WARNING

The thread pool always creates background threads, so if it has work in progress when the last foreground thread in your process exits, that work will not complete, because all background threads will be terminated at that point. If you need to ensure that work being done on the thread pool completes, you must wait for that to happen before allowing all foreground threads to finish.

Launching thread pool work with Task

The usual way to use the thread pool is through the `Task` class. This is part of the Task Parallel Library (discussed in more detail in [“Tasks”](#)), but its basic usage is pretty straightforward, as [Example 16-5](#) shows.

Example 16-5. Running code on the thread pool with a `Task`

```
Task.Run(() => MyThreadEntryPoint("https://oreilly-com.ezproxy.sfp1.org/"));
```

This queues the lambda for execution on the thread pool (which, when it runs, just calls the `MyThreadEntryPoint` method from [Example 16-4](#)). If a thread is available, it will start to run straightaway, but if not, it will wait in a queue until a thread becomes available (either because some other work item in progress completes or because the thread pool decides to add a new thread to the pool).

There are other ways to use the thread pool, the most obvious of which is through the `ThreadPool` class. Its `QueueUserWorkItem` method works in a similar way to `Start`—you pass it a delegate and it will queue the method for execution. This is a lower-level API—it does not provide any direct way to handle completion of the work, nor to chain operations together, so for most cases, the `Task` class is preferable.

Thread creation heuristics

The runtime adjusts the number of threads based on the workload you present. The heuristics it uses are not documented and have changed across releases of .NET, so you should not depend on the exact behavior

I'm about to describe; however, it is useful to know roughly what to expect.

If you give the thread pool only CPU-bound work, in which every method you ask it to execute spends its entire time performing computations and never blocks waiting for I/O to complete, you might end up with one thread for each of the hardware threads in your system (although if the individual work items take long enough, the thread pool might decide to allocate more threads). For example, on the eight-core two-way hyper-threaded computer I'm using as I write this, queuing up a load of CPU-intensive work items initially causes the CLR to create 16 thread pool threads, and as long as the work items complete about once a second or faster, the number of threads mostly stays at that level. (It occasionally goes over that because the runtime will try adding an extra thread from time to time to see what effect this has on throughput, and then it drops back down again.) But if the rate at which the program gets through items drops, the CLR gradually increases the thread count.

If thread pool threads get blocked (e.g., because they're waiting for data from disk or for a response over the network from a server), the CLR increases the number of pool threads more quickly. Again, it starts off with one per hardware thread, but when slow work items consume very little processor time, it will try adding threads to improve throughput.

In either case, the CLR will eventually stop adding threads. The exact default limit varies in 32-bit processes, depending on the version of .NET, although it's typically on the order of 1,000 threads. In 64-bit mode, it appears to default to 32,767. You can change this limit—the `ThreadPool` class has a `SetMaxThreads` method that lets you configure different limits for your process. You may run into other limitations that place a lower practical limit. For example, each thread has its own stack that has to occupy a contiguous range of virtual address space. If each thread gets 1 MB of the process's address space reserved for its stack, by the time you have 1,000 threads, you'll be using 1 GB of address space for stacks alone. Thirty-two-bit processes have only 4 GB of address range, so you might not have space for the number of threads you request. In any case, 1,000 threads is usually more than is helpful, so if it gets that high, this may be

a symptom of some underlying problem that you should investigate. For this reason, if you call `SetMaxThreads`, it will normally be to specify a lower limit—you may find that with some workloads, constraining the number of threads improves throughput by reducing the level of contention for system resources.

`ThreadPool` also has a `SetMinThreads` method. This lets you ensure that the number of threads does not drop below a certain number. This can be useful in applications that work most efficiently with some minimum number of threads and that want to be able to operate at maximum speed instantly, without waiting for the thread pool's heuristics to adjust the thread count.

Thread Affinity and SynchronizationContext

Some objects demand that you use them only from certain threads. This is particularly common with UI code—the WPF, .NET MAUI, and Windows Forms UI frameworks require that UI objects be used from the thread on which they were created. This is called *thread affinity*, and although it is most often a UI concern, it can also crop up in interoperability scenarios—some COM objects have thread affinity.

Thread affinity can make life awkward if you want to write multithreaded code. Suppose you've carefully implemented a multithreaded algorithm that can exploit all of the hardware threads in an end user's computer, significantly improving performance when running on a multicore CPU compared to a single-threaded algorithm. Once the algorithm completes, you may want to present the results to the end user. The thread affinity of UI objects requires you to perform that final step on a particular thread, but your multithreaded code may well produce its final results on some other thread. (In fact, you will probably have avoided the UI thread entirely for the CPU-intensive work, to make sure that the UI remained responsive while the work was in progress.) If you try to update the UI from some random worker thread, the UI framework will throw an exception complaining that you've violated its thread affinity requirements. Somehow, you'll need to pass a message back to the UI thread so that it can display the results.

The runtime libraries provide the `SynchronizationContext` class to help in these scenarios. Its `Current` static property returns an instance of the `SynchronizationContext` class that represents the context in which your code is currently running. For example, in a WPF application, if you retrieve this property while running on a UI thread, it will return an object associated with that thread. You can store the object that `Current` returns and use it from any thread anytime you need to perform further work on the UI thread. [Example 16-6](#) does this so that it can perform some potentially slow work on a thread pool thread and then update the UI back on the UI thread.

Example 16-6. Using the thread pool and then `SynchronizationContext`

```
private void findButton_Click(object sender, RoutedEventArgs e)
{
    SynchronizationContext uiContext = SynchronizationContext.Current!;

    Task.Run(() =>
    {
        string pictures =
            Environment.GetFolderPath(Environment.SpecialFolder.MyPictures);
        var folder = new DirectoryInfo(pictures);
        FileInfo[] allFiles =
            folder.GetFiles("*.jpg", SearchOption.AllDirectories);
        FileInfo? largest =
            allFiles.OrderByDescending(f => f.Length).FirstOrDefault();

        if (largest is not null)
        {
            uiContext.Post(_ =>
            {
                long sizeMB = largest.Length / (1024 * 1024);
                outputTextBox.Text =
                    $"Largest file ({sizeMB}MB) is {largest.FullName}";
            },
            null);
        }
    })
```

```
});  
}
```

This code handles a `Click` event for a button. (It happens to be a WPF application, but `SynchronizationContext` works in exactly the same way in other client-side UI frameworks, such as .NET MAUI.) UI elements raise their events on the UI thread, so when the first line of the click handler retrieves the current `SynchronizationContext`, it will get the context for the UI thread. The code then runs some work on a thread pool thread via the `Task` class. The code looks at every picture in the user's *Pictures* folder, searching for the largest file, so this could take a while. It's a bad idea to perform slow work on a UI thread—UI elements that belong to that thread cannot respond to user input while the UI thread is busy doing something else. So pushing this into the thread pool is a good idea.

The problem with using the thread pool here is that once the work completes, we're on the wrong thread to update the UI. This code updates the `Text` property of a text box, and we'd get an exception if we tried that from a thread pool thread. So, when the work completes, it uses the `SynchronizationContext` object it retrieved earlier and calls its `Post` method. That method accepts a delegate, and it will arrange to invoke that back on the UI thread. (Under the covers, it posts a custom message to the Windows message queue, and when the UI thread's main message processing loop picks up that message, it will invoke the delegate.)

TIP

The `Post` method does not wait for the work to complete. There is a method that will wait, called `Send`, but you should avoid it. Making a worker thread block while it waits for the UI thread to do something can be risky, because if the UI thread is currently blocked waiting for the worker thread to do something, the application will deadlock. `Post` avoids this problem by enabling the worker thread to proceed concurrently with the UI thread.

[Example 16-6](#) retrieves `SynchronizationContext.Current` while it's still on the UI thread, before it starts the thread pool work. This is important because this static property is context sensitive—it returns the context for

the UI thread only while you're on the UI thread. (In fact, it's possible for each window to have its own UI thread in WPF, so it wouldn't be possible to have an API that returns *the* UI thread—there might be several.) If you read this property from a thread pool thread, the context object it returns will not post work to the UI thread.

The `SynchronizationContext` mechanism is extensible, so you can derive your own type from it if you want, and you can call its static `SetSynchronizationContext` method to make your context the current context for the thread. This can be useful in unit testing scenarios—it enables you to write tests to verify that objects interact with the `SynchronizationContext` correctly without needing to create a real UI.

ExecutionContext

The `SynchronizationContext` class has a cousin, `ExecutionContext`. This provides a similar service, allowing you to capture the current context and then use it to run a delegate sometime later in the same context, but it differs in two ways. First, it captures different things. Second, it uses a different approach for reestablishing the context. A `SynchronizationContext` will often run your work on some particular thread, whereas `ExecutionContext` will always use your thread, and it just makes sure that all of the contextual information it has captured is available on that thread. One way to think of the difference is that `SynchronizationContext` does the work in an existing context, whereas `ExecutionContext` brings the contextual information to you.

You retrieve the current context by calling the `ExecutionContext.Capture` method. The execution context does not capture thread-local storage, but it does include any information in the current *logical call context*. You can access this through the `CallContext` class, which provides `LogicalSetData` and `LogicalGetData` methods to store and retrieve name/value pairs, or through the higher-level wrapper `AsyncLocal<T>`. This information is usually associated with the current thread, but if you run code in a captured execution context, it will make information from the logical context available, even if that code runs on some other thread entirely.

.NET uses the `ExecutionContext` class internally whenever long-running work that starts on one thread later ends up continuing on a different thread (as happens with some of the asynchronous patterns described later in this chapter). You may want to use the execution context in a similar way if you write any code that accepts a callback that it will invoke later, perhaps from some other thread. To do this, you call `Capture` to grab the current context, which you can later pass to the `Run` method to invoke a delegate. [Example 16-7](#) shows `ExecutionContext` at work.

Example 16-7. Using `ExecutionContext`

```
public class Defer(Action callback)
{
    private readonly ExecutionContext? _context = ExecutionContext.Capture();

    public void Run()
    {
        if (_context is null) { callback(); return; }
        // When ExecutionContext.Run invokes the lambda we supply as the 2nd
        // argument, it passes that lambda the value we supplied as the 3rd
        // argument to Run. Here we're passing callback, so the lambda has
        // access to the Action we want to invoke. It would have been simpler
        // to write "_ => callback()", but the lambda would then need to
        // capture 'this' to be able to access callback, and that capture
        // would cause an additional allocation. Using the static keyword
        // on the lambda tells the compiler that we intend to avoid capture,
        // so it would report an error if we accidentally used any locals.
        ExecutionContext.Run(
            _context,
            static (cb) => ((Action)cb!)(),
            callback);
    }
}
```

In .NET Framework, a single captured `ExecutionContext` cannot be used on multiple threads simultaneously. Sometimes you might need to invoke multiple different methods in a particular context, and in a multi-threaded environment, you might not be able to guarantee that the previous method has returned before calling the next. For this scenario, `ExecutionContext` provides a `CreateCopy` method that generates a copy

of the context, enabling you to make multiple simultaneous calls through equivalent contexts. In .NET, `ExecutionContext` is immutable, meaning this restriction no longer applies, and `CreateCopy` just returns its `this` reference.

Synchronization

Sometimes you will want to write multithreaded code in which multiple threads have access to the same state. For example, in [Chapter 5](#), I suggested that a server could use a `Dictionary<TKey, TValue>` as part of a cache to avoid duplicating work when it receives multiple similar requests. While this sort of caching can offer significant performance benefits in some scenarios, it presents a challenge in a multithreaded environment. (And if you're working on server code with demanding performance requirements, you will most likely need more than one thread to handle requests.) The Thread Safety section of the documentation for the `Dictionary<TKey, TValue>` class says this:

A `Dictionary<TKey, TValue>` can support multiple readers concurrently, as long as the collection is not modified. Even so, enumerating through a collection is intrinsically not a thread-safe procedure. In the rare case where an enumeration contends with write accesses, the collection must be locked during the entire enumeration. To allow the collection to be accessed by multiple threads for reading and writing, you must implement your own synchronization.

This is better than we might expect—the vast majority of types in the runtime libraries simply don't support multithreaded use of instances at all. Most types support multithreaded use at the class level, but individual instances must be used one thread at a time. `Dictionary<TKey, TValue>` is more generous: it explicitly supports multiple concurrent readers, which sounds good for our caching scenario. However, when modifying a collection, not only must we ensure that we do not try to change it from multiple threads simultaneously, but also we must not have any read operations in progress while we do so.

The other generic collection classes make similar guarantees (unlike most other classes in the library). For example, `List<T>`, `Queue<T>`, `Stack<T>`, `SortedDictionary<TKey, TValue>`, `HashSet<T>`, and `SortedSet<T>` all support concurrent read-only use. (Again, if you modify any instance of these collections, you must make sure that no other threads are either modifying or reading from the same instance at the same time.) Of course, you should always check the documentation before attempting multithreaded use of any type.² Be aware that the generic collection interface types make no thread safety guarantees—although `List<T>` supports concurrent readers, not all implementations of `IList<T>` will. (For example, imagine an implementation that wraps something potentially slow, such as the contents of a file. It might make sense for this wrapper to cache data to make read operations faster. Reading an item from such a list could change its internal state, so reads could fail when performed simultaneously from multiple threads if the code did not take steps to protect itself.)

If you can arrange never to have to modify a data structure while it is in use from multithreaded code, the support for concurrent access offered by many of the collection classes may be all you need. But if some threads will need to modify shared state, you will need to coordinate access to that state. To enable this, .NET provides various synchronization mechanisms that you can use to ensure that your threads take it in turns to access shared objects when necessary. In this section, I'll describe the most commonly used ones.

Monitors and the lock Keyword

The first option to consider for synchronizing multithreaded use of shared state is the `Monitor` class. This is popular because it is efficient, it offers a straightforward model, and C# provides direct language support, making it very easy to use. [Example 16-8](#) shows a class that uses the `lock` keyword (which in turn uses the `Monitor` class) anytime it either reads or modifies its internal state. This ensures that only one thread will be accessing that state at any one time.

Example 16-8. Protecting state with `lock`

```
public class SaleLog
{
    private readonly object _sync = new();
    private decimal _total;
    private readonly List<string> _saleDetails = [];

    public decimal Total
    {
        get { lock (_sync) { return _total; } }
    }

    public void AddSale(string item, decimal price)
    {
        string details = $"{item} sold at {price}";
        lock (_sync)
        {
            _total += price;
            _saleDetails.Add(details);
        }
    }

    public string[] GetDetails(out decimal total)
    {
        lock (_sync)
        {
            total = _total;
            return _saleDetails.ToArray();
        }
    }
}
```

To use the `lock` keyword, you provide a reference to an object and a block of code. The C# compiler generates code that will cause the CLR to ensure that no more than one thread is inside a `lock` block for that object at any one time. Suppose you created a single instance of this `SaleLog` class, and on one thread you called the `AddSale` method, while on another thread you called `GetDetails` at the same time. Both threads will reach `lock` statements, passing in the same `_sync` field. Whichever thread happens to get there first will be allowed to run the block follow-

ing the `lock`. The other thread will be made to wait—it won't be allowed to enter its `lock` block until the first thread leaves its `lock` block.

The `SaleLog` class only ever uses any of its fields from inside a `lock` block using the `_sync` argument. This ensures that all access to fields is serialized (in the concurrency sense—that is, threads get to access fields one at a time, rather than all piling in simultaneously). When the `GetDetails` method reads from both the `_total` and `_saleDetails` fields, it can be confident that it's getting a coherent view—the total will be consistent with the current contents of the list of sales details, because the code that modifies these two pieces of data does so within a single `lock` block. This means that updates will appear to be atomic from the point of view of any other `lock` block using `_sync`.

It may look excessive to use a `lock` block even for the `get` accessor that returns the total. However, `decimal` is a 128-bit value, so access to data of this type is not intrinsically atomic—without that `lock`, it would be possible for the returned value to be made up of a mixture of two or more values that `_total` had at different times. (For example, the bottom 64 bits might be from an older value than the top 64 bits.) This is often described as a *torn read*. The CLR guarantees atomic reads and writes only for data types whose size is no larger than 4 bytes, and also for references, even on a platform where those are larger than 4 bytes. (It guarantees this only for naturally aligned fields, but in C#, fields will always be aligned unless you have deliberately misaligned them for interop purposes.)

A subtle but important detail of [Example 16-8](#) is that whenever it returns information about its internal state, it returns a copy. The `Total` property's type is `decimal`, which is a value type, and values are always returned as copies. But when it comes to the list of entries, the `GetDetails` method calls `ToArray`, which will build a new array containing a copy of the list's current contents. It would be a mistake to return the reference in `_saleDetails` directly, because that would enable code outside of the `SalesLog` class to access and modify the collection without using `lock`. We need to ensure that all access to that collection is

synchronized, and we lose the ability to do that if our class hands out references to its internal state.

TIP

If you write code that performs some multithreaded work that eventually comes to a halt, it's OK to share references to the state after the work has stopped. But if multithreaded modifications to an object are ongoing, you need to ensure that all use of that object's state is protected.

The `lock` keyword accepts any object reference, so you might wonder why I've created an object specially—couldn't I have passed `this` instead? That would have worked, but the problem is that your `this` reference is not private—it's the same reference by which external code uses your object. Using a publicly visible feature of your object to synchronize access to private state is imprudent; some other code could decide that it's convenient to use a reference to your object as the argument to some completely unrelated `lock` blocks. In this case, it probably wouldn't cause a problem, but with more complex code, it could tie conceptually unrelated pieces of concurrent behavior together in a way that might cause performance problems or even deadlocks. Thus, it's usually better to code defensively and use something that only your code has access to as the `lock` argument. Of course, I could have used the `_saleDetails` field because that refers to an object that only my class has access to. However, even if you code defensively, you should not assume that other developers will, so in general, it's safer to avoid using an instance of a class you didn't write as the argument for a `lock`, because you can never be certain that it isn't using its `this` reference for its own locking purposes.

The fact that you can use any object reference is a bit of an oddity in any case. Most of .NET's synchronization mechanisms use an instance of some distinct type as the point of reference for synchronization. (For example, if you want reader/writer locking semantics, you use an instance of the `ReaderWriterLockSlim` class, not just any old object.) The `Monitor` class (which is what `lock` uses) is an exception that dates back to an old requirement for a degree of compatibility with Java (which has a similar

locking primitive). This is not relevant to modern .NET development, so this feature is now just a historical peculiarity. Using a distinct object whose only job is to act as a `lock` argument adds minimal overhead (compared to the costs of locking in the first place) and tends to make it easier to see how synchronization is being managed.

NOTE

You cannot use a value type as an argument for `lock`. C# prevents this, and with good reason. The compiler performs an implicit conversion to `object` on the `lock` argument, which for reference types doesn't require the CLR to do anything at runtime. But when you convert a value type to a reference of type `object`, a box needs to be created. That box would be the argument to `lock`, and that would be a problem, because you get a new box every time you convert a value to an `object` reference. So, each time you ran a `lock`, it would get a different object, meaning there would be no synchronization in practice. This is why the compiler prevents you from trying.

How the `lock` keyword expands

Each `lock` block turns into code that does three things: first, it calls `Monitor.Enter`, passing the argument you provided to `lock`. Then it attempts to run the code in the block. Finally, it will usually call `Monitor.Exit` once the block finishes. But it's not entirely straightforward, thanks to exceptions. The code will still call `Monitor.Exit` if the code you put in the block throws an exception, but it needs to handle the possibility that `Monitor.Enter` itself threw, which would mean that the thread does not own the lock and should therefore not call `Monitor.Exit`. [Example 16-9](#) shows what the compiler makes of the `lock` block in the `GetDetails` method in [Example 16-8](#).

Example 16-9. How `lock` blocks expand

```
bool lockWasTaken = false;
object temp = _sync;
try
{
    Monitor.Enter(temp, ref lockWasTaken);
```

```

        {
            total = _total;
            return _saleDetails.ToArray();
        }
    }
    finally
    {
        if (lockWasTaken)
        {
            Monitor.Exit(temp);
        }
    }
}

```

`Monitor.Enter` is the API that does the work of discovering whether some other thread already has the lock, and if so, making the current thread wait. If this returns at all, it normally succeeds. (It might deadlock, in which case it will never return.) There is a small possibility of failure caused by an exception, e.g., due to running out of memory. That would be unusual, but the generated code takes it into account nonetheless—this is the purpose of the slightly roundabout-looking code for the `lockWasTaken` variable. (In practice, the compiler will make that a hidden variable without an accessible name, by the way. I've named it to show what's happening here.) The `Monitor.Enter` method guarantees that acquisition of the lock will be atomic with updating the flag indicating whether the lock was taken, ensuring that the `finally` block will attempt to call `Exit` if and only if the lock was acquired.

`Monitor.Exit` tells the CLR that we no longer need exclusive access to whatever resources we're synchronizing access to, and if any other threads are waiting inside `Monitor.Enter` for the object in question, this will enable one of them to proceed. The compiler puts this inside a `finally` block to ensure that whether you exit from the block by running to the end, returning from the middle, or throwing an exception, the lock will be released.

The fact that the `lock` block calls `Monitor.Exit` on an exception is a double-edged sword. On the one hand, it reduces the chances of deadlock by ensuring that locks are released on failure. On the other hand, if an exception occurs while you're in the middle of modifying some shared state,

the system may be in an inconsistent state; releasing locks will allow other threads access to that state, possibly causing further problems. In some situations, it might have been better to leave locks locked in the case of an exception—a deadlocked process might do less damage than one that plows on with corrupt state. A more robust strategy is to write code that guarantees consistency in the face of exceptions, either by rolling back any changes it has made if an exception prevents a complete set of updates or by arranging to change state in an atomic way (e.g., by putting the new state into a whole new object and substituting that for the previous one only once the updated object is fully initialized). But that's beyond what the compiler can automate for you.

Waiting and notification

The `Monitor` class can do more than just ensure that threads take it in turns. It provides a way for threads to sit and wait for a notification from some other thread. If a thread has acquired the monitor for a particular object, it can call `Monitor.Wait`, passing in that object. This has two effects: it releases the monitor and causes the thread to block. It will block until some other thread calls `Monitor.Pulse` or `PulseAll` for the same object; a thread must have the monitor to be able to call either of these methods. (`Wait`, `Pulse`, and `PulseAll` all throw an exception if you call them while not holding the relevant monitor.)

If a thread calls `Pulse`, this enables one thread waiting in `Wait` to wake up. Calling `PulseAll` enables all of the threads waiting on that object's monitor to run. In either case, `Monitor.Wait` reacquires the monitor before returning, so even if you call `PulseAll`, the threads will wake up one at a time—a second thread cannot emerge from `Wait` until the first thread to do so relinquishes the monitor. In fact, no threads can return from `Wait` until the thread that called `Pulse` or `PulseAll` relinquishes the lock.

[Example 16-10](#) uses `Wait` and `Pulse` to provide a wrapper around a `Queue<T>` that causes the thread that retrieves items from the queue to wait if the queue is empty. (This is for illustration only—if you want this

sort of queue, you don't have to write your own. Use the built-in `BlockingCollection<T>` or the types in `System.Threading.Channels`.)

Example 16-10. `Wait` and `Pulse`

```
public class MessageQueue<T>
{
    private readonly object _sync = new();

    private readonly Queue<T> _queue = new();

    public void Post(T message)
    {
        lock (_sync)
        {
            bool wasEmpty = _queue.Count == 0;
            _queue.Enqueue(message);
            if (wasEmpty)
            {
                Monitor.Pulse(_sync);
            }
        }
    }

    public T Get()
    {
        lock (_sync)
        {
            while (_queue.Count == 0)
            {
                Monitor.Wait(_sync);
            }
            return _queue.Dequeue();
        }
    }
}
```

This example uses the monitor in two ways. It uses it through the `lock` keyword to ensure that only one thread at a time uses the `Queue<T>` that holds queued items. But it also uses waiting and notification to enable the thread that consumes items to block efficiently when the queue is empty,

and for any thread that adds new items to the queue to wake up the blocked reader thread.

Timeouts

Whether you are waiting for a notification or just attempting to acquire the lock, it's possible to specify a timeout, indicating that if the operation doesn't succeed within the specified time, you would like to give up. For lock acquisition, you use a different method, `TryEnter`, but when waiting for notification, you just use a different overload. (There's no compiler support for this, so you won't be able to use the `lock` keyword.) In both cases, you can pass either an `int` representing the maximum time to wait, in milliseconds, or a `TimeSpan` value. Both return a `bool` indicating whether the operation succeeded.

You could use this to avoid deadlocking the process, but if your code does fail to acquire a lock within the timeout, this leaves you with the problem of deciding what to do about that. If your application is unable to acquire a lock it needs, then it can't just do whatever work it was going to do regardless. Termination of the process may be the only realistic option, because deadlock is usually a symptom of a bug, so if it occurs, your process may already be in a compromised state. That said, some developers take a less-than-rigorous approach to lock acquisition and may regard deadlock as being normal. In this case, it might be viable to abort whatever operation you were trying and either retry the work later or just log a failure, abandon this particular operation, and carry on with whatever else the process was doing. But that may be a risky strategy.

Other Synchronization Primitives

Although the `lock` keyword should be your default choice for protecting shared data in multithreaded scenarios, the .NET runtime libraries offer many specialized synchronization primitives that can be a better fit in certain cases. [Table 16-1](#) describes the scenarios for which these are intended.

Table 16-1. Specialized synchronization types

Type	Usage
Barrier	Enables multiple threads to coordinate their work in phases.
CountdownEvent	Enables threads to wait until the CountdownEvent has been signaled some specific number of times.
ManualResetEvent, AutoResetEvent, and EventWaitHandle	Enable one thread to signal to other threads when something of interest has happened. Support cross-process notifications on Windows.
ManualResetEventSlim	Alternative to ManualResetEvent that does not support cross-process notification, but which has lower overheads when wait times are likely to be very short.
Mutex	Exclusive access similar to Monitor. Higher overhead, but with cross-process support on all operating systems.
ReaderWriterLockSlim	Higher overhead than Monitor, but can be better if locks are held for a long time and modifications are rare, because this allows concurrent readers, granting exclusive access only during writes.
SpinLock	Exclusive locking (just like lock and Monitor) that might enable lower memory usage; requires great care because subtle mistakes can make this more expensive than simpler solutions.
Semaphore	Enables a bounded level of concurrency—like a Monitor where you can configure the number of threads that are allowed to possess it simultaneously. Supports cross-process use on Windows.
SemaphoreSlim	Alternative to Semaphore that does not support cross-process notification, but which has lower overheads when wait times are likely to be very short.

Interlocked

The `Interlocked` class supports concurrent access to shared data, but it is not a synchronization primitive. Instead, it defines static methods that provide atomic forms of various simple operations.

For example, it provides `Increment`, `Decrement`, and `Add` methods, with overloads supporting `int` and `long` values. (These are all similar—incrementing or decrementing is just addition by 1 or -1.) Addition involves reading a value from some storage location, calculating a modified value, and storing that back in the same storage location, and if you use normal C# operators to do this, things can go wrong if multiple threads try to modify the same location simultaneously. If the value is initially `0`, and two threads both read that value in quick succession, and if both then add 1 and store the result back, they will both end up writing back `1`. Two threads attempted to increment the value, but it went up only by one. The `Interlocked` form of these operations prevents this sort of overlap.

`Interlocked` also offers various methods for swapping values. The `Exchange` method takes two arguments: a reference to a value and a value. This returns the value currently in the location referred to by the first argument and also overwrites that location with the value supplied as a second argument, and it performs these two steps as a single atomic operation. There are overloads supporting `int`, `uint`, `long`, `ulong`, `object`, `float`, `double`, `nint`, and `nuint`. There is also a generic `Exchange<T>`, where `T` can be any reference type. There is a variant called `CompareExchange`. As with `Exchange`, it takes a reference to some variable you wish to modify, and the value you want to replace it with, but it also takes a third argument: the value you think is already in the storage location. If the value in the storage location does not match the expected value, this method will not change the storage location. (It still returns whatever value was in that storage location, whether it modifies it or not.) This is often used to discover when some other thread has been using the variable at the same time as us, and to avoid concurrent modifications.

The simplest `Interlocked` operation is the `Read` method. This takes a `ref long` or `ref ulong` and reads the value atomically with respect to any other operations on the same variable that you perform through `Interlocked`. This enables you to read 64-bit values safely—in general, the CLR does not guarantee that 64-bit reads will be atomic. (In a 64-bit process, they will be unless you’ve taken deliberate steps to misalign data, which may sometimes be necessary to interoperate with unmanaged code. But 64-bit reads usually aren’t atomic on 32-bit architectures. You need to use `Interlocked.Read` to ensure atomicity.) There are no overloads for 32-bit values, because reading and writing those is always atomic.

The operations supported by `Interlocked` correspond to the atomic operations that most CPUs can support more or less directly. (Some CPU architectures support all the operations innately, while others support only the compare and exchange, building everything else up out of that. But in any case, these operations are at most a few instructions.) This means they are reasonably efficient. They are considerably more costly than performing equivalent noninterlocked operations with ordinary code, because atomic CPU instructions need to coordinate across all CPU cores (and across all CPU chips in computers that have multiple physically separate CPUs installed) to guarantee atomicity. Nonetheless, they incur a fraction of the cost you pay when a `lock` statement ends up blocking the thread at the OS level.

These sorts of operations are sometimes described as *lock free*. This is not entirely accurate—the computer does acquire locks very briefly at a fairly low level in the hardware. Atomic read-modify-write operations effectively acquire an exclusive lock on the computer’s memory for two bus cycles. However, no OS locks are acquired, the scheduler does not need to get involved, and the locks are held for an extremely short duration—often for just one machine code instruction. More significantly, the highly specialized and low-level form of locking used here does not permit holding on to one lock while waiting to acquire another—code can lock only one thing at a time. This means that this sort of operation will not deadlock. However, the simplicity that rules out deadlocks cuts both ways.

The downside of interlocked operations is that the atomicity applies only to extremely simple operations. It's very hard to build more complex logic in a way that works correctly in a multithreaded environment using just `Interlocked`. It's easier and considerably less risky to use the higher-level synchronization primitives, because those make it fairly easy to protect more complex operations rather than just individual calculations. You would typically use `Interlocked` only in extremely performance-sensitive work, and even then, you should measure carefully to verify that it's having the effect you hope—sometimes *clever* use of `Interlocked` ends up costing you more than you expect.

One of the biggest challenges with writing correct code when using low-level atomic operations is that you may encounter problems caused by the way a CPU's cache works. Work done by one thread may not become visible instantly to other threads, and in some cases, memory access may not necessarily occur in the order that your code specifies. Using higher-level synchronization primitives sidesteps these issues by enforcing certain ordering constraints, but if you decide instead to use `Interlocked` to build your own synchronization mechanisms, you will need to understand the memory model that .NET defines for when multiple threads access the same memory simultaneously, and you will typically need to use either the `MemoryBarrier` method defined by the `Interlocked` class or the various methods defined by the `Volatile` class to ensure correctness. This is beyond the scope of this book, and it's also a really good way to write code that looks like it works but turns out to go wrong under heavy load (i.e., when it probably matters most), so these sorts of techniques are rarely worth the cost. Stick with the other mechanisms I've discussed in this chapter unless you really have no alternative.

Lazy Initialization

When you need an object to be accessible from multiple threads, if it's possible for that object to be immutable (i.e., its fields never change after construction), you can often avoid the need for synchronization. It is always safe for multiple threads to read from the same location simultaneously—trouble sets in only if the data needs to change. However, there is one challenge: When and how do you initialize the shared object? One so-

lution might be to store a reference to the object in a static field initialized from a static constructor or a field initializer—the CLR guarantees to run the static initialization for any class just once. However, this might cause the object to be created earlier than you want. If you perform too much work in static initialization, this can have an adverse effect on how long it takes your application to start running.

You might want to wait until the object is first needed before initializing it. This is called *lazy initialization*. This is not particularly hard to achieve—you can just check a field to see if it's `null` and initialize it if not, using `lock` to ensure that only one thread gets to construct the value. However, this is an area in which developers seem to have a remarkable appetite for showing how clever they are, with the potentially undesirable corollary of demonstrating that they're not as clever as they think they are.

The `lock` keyword works fairly efficiently, but it's possible to do better by using `Interlocked`. However, the subtleties of memory access re-ordering on multiprocessor systems make it easy to write code that runs quickly, looks clever, and doesn't always work. To avert this recurring problem, .NET provides two classes to perform lazy initialization without using `lock` or other potentially expensive synchronization primitives. The easiest to use is `Lazy<T>`.

Lazy<T>

The `Lazy<T>` class provides a `Value` property of type `T`, and it will not create the instance that `Value` returns until the first time something reads the property. By default, `Lazy<T>` will use the no-arguments constructor for `T`, but you can supply your own method for creating the instance.

`Lazy<T>` is able to handle race conditions for you. In fact, you can configure the level of multithreaded protection you require. Since lazy initialization can also be useful in single-threaded environments, you can disable multithreaded support entirely (by passing `LazyThreadSafetyMode.None` as a constructor argument). But for multithreaded environments, you can choose between the other two modes in the `LazyThreadSafetyMode` enumeration.

These determine what happens if multiple threads all try to read the `Value` property for the first time more or less simultaneously. `PublicationOnly` does not attempt to ensure that only one thread creates an object—it only applies any synchronization at the point at which a thread finishes creating an object. The first thread to complete construction or initialization gets to supply the object, and the ones produced by any other threads that had started initialization are all discarded. Once a value is available, all further attempts to read `Value` will just return that.

If you choose `ExecutionAndPublication`, only a single thread will be allowed to attempt construction. That may seem less wasteful, but `PublicationOnly` offers a potential advantage: because it avoids holding any locks during initialization, you are less likely to introduce deadlock bugs if the initialization code itself attempts to acquire any locks. `PublicationOnly` also handles errors differently. If the first initialization attempt throws an exception, other threads that had begun a construction attempt are given a chance to complete, whereas with `ExecutionAndPublication`, if the one and only attempt to initialize fails, the exception is retained and will be thrown each time any code reads `Value`.

LazyInitializer

The other class supporting lazy initialization is `LazyInitializer`. This is a static class, and you use it entirely through its static generic methods. It is marginally more complex to use than `Lazy<T>`, but it avoids the need to allocate an extra object in addition to the lazily allocated instance you require. [Example 16-11](#) shows how to use it.

Example 16-11. Using `LazyInitializer`

```
public class Cache<T>
{
    private static Dictionary<string, T>? _d;

    public static IDictionary<string, T> Dictionary =>
        LazyInitializer.EnsureInitialized(ref _d);
}
```

If the field is null, the `EnsureInitialized` method constructs an instance of the argument type—`Dictionary<string, T>`, in this case. Otherwise, it will return the value already in the field. There are some other overloads. You can pass a callback, much as you can to `Lazy<T>`. You can also pass a `ref bool` argument, which it will inspect to discover whether initialization has already occurred (and it sets this to `true` when it performs initialization).

A static field initializer would have given us the same once-and-once-only initialization but might have ended up running far earlier in the process's lifetime. In a more complex class with multiple fields, static initialization might even cause unnecessary work, because it happens for the entire class, so you might end up constructing objects that don't get used. This could increase the amount of time it takes for an application to start up. `LazyInitializer` lets you initialize individual fields as and when they are first used, ensuring that you do only work that is needed.

Other Class Library Concurrency Support

The `System.Collections.Concurrent` namespace defines various collections that make more generous guarantees in the face of multithreading than the usual collections, meaning you may be able to use them without needing any other synchronization primitives. Take care, though—as always, even though individual operations may have well-defined behavior in a multithreaded world, that doesn't necessarily help you if the operation you need to perform involves multiple steps. You may still need coordination at a broader scope to guarantee consistency. But in some situations, the concurrent collections may be all you need.

Unlike the nonconcurrent collections, `ConcurrentDictionary`, `ConcurrentBag`, `ConcurrentStack`, and `ConcurrentQueue` all support modification of their contents even while enumeration (e.g., with a `foreach` loop) of those contents is in progress. The dictionary provides a live enumerator, in the sense that if values are added or removed while you're in the middle of enumerating, the enumerator might show you some of the added items and it might not show you the removed items. It makes no firm guarantees, not least because with multithreaded code,

when two things happen on two different threads, it's not always entirely clear which happened first—the laws of relativity mean that it may depend on your point of view.

This means that it's possible for an enumerator to seem to return an item after that item was removed from the dictionary. The bag, stack, and queue take a different approach: their enumerators all take a snapshot and iterate over that, so a `foreach` loop will see a set of contents that is consistent with what was in the collection at some point in the past, even though it may since have changed.

As I already mentioned in [Chapter 5](#), the concurrent collections present APIs that have similarities to their nonconcurrent counterparts but with some additional members to support atomic addition and removal of items. For example, `ConcurrentDictionary` offers a `GetOrAdd` method that returns an existing entry if one exists and adds a new entry otherwise.

Another part of the runtime libraries that can help you deal with concurrency without needing to make explicit use of synchronization primitives is Rx (the subject of [Chapter 11](#)). It offers various operators that can combine multiple asynchronous streams together into a single stream. These manage concurrency issues for you—remember that any single observable will provide observers with items one at a time.

Rx takes the necessary steps to ensure that it stays within these rules even when it combines inputs from numerous individual streams that are all producing items concurrently. As long as all the sources stick to the rules, Rx will never ask an observer to deal with more than one thing at a time.

The `System.Threading.Channels` NuGet package offers types that support producer/consumer patterns, in which one or more threads generate data, while other threads consume that data. You can choose whether channels are buffered, enabling producers to get ahead of consumers, and if so, by how much. (The `BlockingCollection<T>` in `System.Collections.Concurrent` also offers this kind of service. However, it is less flexible, and it does not support the `await` keyword described in [Chapter 17](#).)

Finally, in multithreaded scenarios it is worth considering the immutable collection classes, which I described in [Chapter 5](#). These support concurrent access from any number of threads, and because they are immutable, the question of how to handle concurrent write access never arises. Obviously, immutability imposes considerable constraints, but if you can find a way to work with these types (and remember, the built-in `string` type is immutable, so working with immutable data is common), they can be very useful in some concurrent scenarios.

Tasks

Earlier in this chapter, I showed how to use the `Task` class to launch work in the thread pool. This class is more than just a wrapper for the thread pool. `Task` and the related types that form the Task Parallel Library (TPL) can handle a wider range of scenarios. Tasks are particularly important because C#'s asynchronous language features (which are the topic of [Chapter 17](#)) are able to work with these directly. A great many APIs in the runtime libraries offer task-based asynchronous operation.

Although tasks are the preferred way to use the thread pool, they are not just about multithreading. The basic abstractions are more flexible than that.

The `Task` and `Task<T>` Classes

There are two classes at the heart of the TPL: `Task` and a class that derives from it, `Task<T>`. The `Task` base class represents some work that may take some time to complete. `Task<T>` extends this to represent work that produces a result (of type `T`) when it completes. (The nongeneric `Task` does not produce any result. It's the asynchronous equivalent of a `void` return type.) Notice that these are not concepts that necessarily involve threads.

Most I/O operations can take a while to complete, and in most cases, the runtime libraries provide task-based APIs for them. [Example 16-12](#) uses an asynchronous method to fetch the content of a web page as a string.

Since the `HttpClient` class cannot return the string immediately—it might take a while to download the page—it returns a task instead.

Example 16-12. Task-based web download

```
var w = new HttpClient();  
Task<string> webGetTask = w.GetStringAsync("https://endjin.com/");
```

NOTE

Most task-based APIs follow a naming convention in which they end in `Async`, and if there's a corresponding synchronous API, it will have the same name but without the `Async` suffix. For example, the `Stream` class (see [Chapter 15](#)) has a `Write` method, and that method is synchronous (i.e., it waits until it finishes its work before returning). It also offers `WriteAsync`, which, being asynchronous, returns without waiting for its work to complete. It returns a `Task` to represent the work; this convention is called the *Task-based Asynchronous Pattern* (TAP).

That `GetStringAsync` method does not wait for the download to complete, so it returns almost immediately. To perform the download, the computer has to send a message to the relevant server, and then it must wait for a response. Once the request is on its way, there's no work for the CPU to do until the response comes in, meaning that this operation does not need to involve a thread for the majority of the time that the request is in progress. So this method does not wrap some underlying synchronous version of the API in a call to `Task.Run`. And with classes that offer I/O APIs in both forms, such as `Stream`, the synchronous versions are often wrappers around a fundamentally asynchronous implementation: when you call a blocking API to perform I/O, it will typically perform an asynchronous operation under the covers and then just block the calling thread until that work completes. And even in cases where it's nonasynchronous all the way down to the OS—e.g., the `FileStream` can use nonasynchronous operating system file APIs to implement `Read` and `Write`—I/O in the OS kernel is typically asynchronous in nature.

So, although the `Task` and `Task<T>` classes make it very easy to produce tasks that work by running methods on thread pool threads, they are also

able to represent fundamentally asynchronous operations that do not require the use of a thread for most of their duration. Although it's not part of the official terminology, I describe this kind of operation as a *threadless task*, to distinguish it from tasks that run entirely on thread pool threads.

ValueTask and ValueTask<T>

`Task` and `Task<T>` are pretty flexible, and not just because they can represent both thread-based and threadless operations. As you'll see, they offer several mechanisms for discovering when the work they represent completes, including the ability to combine multiple tasks into one. Multiple threads can all wait on the same task simultaneously. You can write caching mechanisms that repeatedly hand out the same task, even long after the task completes. This is all very convenient, but it means that these task types also have some overheads. For more constrained cases, .NET defines less flexible `ValueTask` and `ValueTask<T>` types that are more efficient in certain circumstances.

The most important difference between these types and their ordinary counterparts is that `ValueTask` and `ValueTask<T>` are value types. This is significant in performance-sensitive code because it can reduce the number of objects that code allocates, reducing the amount of time an application spends performing garbage collection work. You might be thinking that the context switching costs typically involved with concurrent work are likely to be high enough that the cost of an object allocation will be the least of your concerns when dealing with asynchronous operations. And while this is often true, there's one very important scenario where the GC overhead of `Task<T>` can be problematic: operations that sometimes run slowly but usually don't.

It is very common for I/O APIs to perform buffering to reduce the number of calls into the OS. If you write a few bytes into a `Stream`, it will typically put those into a buffer and wait until either you've written enough data to make it worth sending it to the OS or you've explicitly called `Flush`. And it's also common for reads to be buffered—if you read a single byte from a file, the OS will typically have to read an entire sector from the drive (usually at least 4 KB), and that data usually gets saved somewhere

in memory so that when you ask for the second byte, no more I/O needs to happen. The practical upshot is that if you write a loop that reads data from a file in relatively small chunks (e.g., one line of text at a time), the majority of read operations will complete straightaway because the data being read has already been fetched.

In these cases where the overwhelming majority of calls into asynchronous APIs complete immediately, the GC overheads of creating task objects can become significant. This is why `ValueTask` and `ValueTask<T>` were introduced. (These are built into .NET, and .NET Standard 2.1. On .NET Framework, you can get them via the `System.Threading.Tasks.Extensions` NuGet package.) These make it possible for potentially asynchronous operations to complete immediately without needing to allocate any objects. In cases where immediate completion is not possible, these types end up being wrappers for `Task` or `Task<T>` objects, at which point the overheads return, but in cases where only a small fraction of calls need to do that, these types can offer significant performance boosts, particularly in code that uses the low-allocation techniques described in [Chapter 18](#).

The nongeneric `ValueTask` is rarely used, because asynchronous operations that produce no result can just return the `Task.CompletedTask` static property, which provides a reusable task that is already in the completed state, avoiding any GC overhead. But tasks that need to produce a result generally can't reuse existing tasks. (There are some exceptions: the runtime libraries will often use cached precompleted tasks for `Task<bool>`, because there are only two possible outcomes. But for `Task<int>`, there's no practical way to maintain a list of precompleted tasks for every possible result.)

These value task types have some constraints. They are single use: unlike `Task` and `Task<T>`, you must not store these types in a dictionary or a `Lazy<T>` to provide a cached asynchronous value. It is an error to attempt to retrieve the `Result` of a `ValueTask<T>` before it has completed. It is also an error to retrieve the `Result` more than once. In general, you should use a `ValueTask` or `ValueTask<T>` with exactly one `await` operation (as described in [Chapter 17](#)) and then never use it again.

(Alternatively, if necessary, you can escape these restrictions by calling its `AsTask` method to obtain a full `Task`, or `Task<T>` with all the corresponding overheads, at which point you should not do anything more with the value task.)

Because the value type tasks were introduced many years after the TPL first appeared, class libraries often use `Task<T>` where you might expect to see a `ValueTask<T>`. For example, the `Stream` class's `ReadAsync` methods are all prime candidates because buffering means these are likely to complete immediately a high proportion of the time. However, most of `Stream`'s asynchronous methods were defined long before `ValueTask<T>` existed, so they mostly return `Task<T>`. The recently added `ReadAsync` overload that accepts a `Memory<byte>` instead of a `byte[]` does return a `ValueTask<T>`, though, and more generally, where APIs have been augmented to add support for the new memory-efficient techniques described in [Chapter 18](#), these will usually return `ValueTask<T>`. And if you're in a performance-sensitive world where the GC overhead of a task is significant, you will likely want to be using those techniques in any case.

Task creation options

Instead of using `Task.Run`, you can get more control over certain aspects of a new thread-based task by creating it with the `StartNew` method of either `Task.Factory` or `Task<T>.Factory`, depending on whether your task needs to return a result. Some overloads of `StartNew` take an argument of the `enum` type `TaskCreationOptions`, which provides some control over how the TPL schedules the task.

The `PreferFairness` flag asks to run the task after any tasks that have already been scheduled. By default, the thread pool normally runs the most recently added tasks first (a last-in, first-out, or LIFO, policy) because this tends to make more efficient use of the CPU cache.

The `LongRunning` flag warns the TPL that the task may run for a long time. By default, the TPL's scheduler optimizes for relatively short work items—anything up to a few seconds. This flag indicates that the work

might take longer than that, in which case the TPL may modify its scheduling. If there are too many long-running tasks, they might use up all the threads, and even though some of the queued work items might be for much shorter pieces of work, those will still take a long time to finish, because they'll have to wait in line behind the slow work before they can even start. But if the TPL knows which items are likely to run quickly and which are likely to be slower, it can prioritize them differently to avoid such problems.

The other `TaskCreationOptions` settings relate to parent/child task relationships and schedulers, which I'll describe later.

Task status

A task goes through a number of states in its lifetime, and you can use the `Task` class's `Status` property to discover where it has gotten to. This returns a value of the `enum` type `TaskStatus`. If a task completes successfully, the property will return the enumeration's `RanToCompletion` value. If the task fails, it will be `Faulted`. If you cancel a task using the technique shown in [“Cancellation”](#), the status will then be `Canceled`.

There are several variations on a theme of “in progress,” of which `Running` is the most obvious—it means that some thread is currently executing the task. A task representing I/O doesn't typically require a thread while it is in progress, so it never enters that state—it starts in the `WaitingForActivation` state and then typically transitions directly to one of the three final states (`RanToCompletion`, `Faulted`, or `Canceled`). A thread-based task can also be in this `WaitingForActivation` state but only if something is preventing it from running, which would typically happen if you set it up to run only when some other task completes (which I'll show how to do shortly). A thread-based task may also be in the `WaitingToRun` state, which means that it's in a queue waiting for a thread pool thread to become available. It's possible to establish parent/child relationships between tasks, and a parent that has already finished but that created some child tasks that are not yet complete will be in the `WaitingForChildrenToComplete` state.

Finally, there's the `Created` state. You don't see this very often, because it represents a thread-based task that you have created but have not yet asked to run. You'll never see this with a task created using the task factory's `StartNew` method, or with `Task.Run`, but you will see this if you construct a new `Task` directly.

The level of detail in the `TaskStatus` property may be too much most of the time, so the `Task` class defines various simpler `bool` properties. If you want to know only whether the task has no more work to do (and don't care whether it succeeded, failed, or was canceled), there's the `IsCompleted` property. `IsCompletedSuccessfully` tells you whether it completed without failure or cancellation. If you want to check specifically for failure or cancellation, use `IsFaulted` or `IsCanceled`.

Retrieving the result

Suppose you've got a `Task<T>`, either from an API that provides one or by creating a thread-based task that returns a value. If the task completes successfully, you are likely to want to retrieve its result, which you can get from the `Result` property. So the task created by [Example 16-12](#) makes the web page content available in `webGetTask.Result`.

If you try to read the `Result` property before the task completes, it will block your thread until the result is available. (If you have a plain `Task`, which does not return a result, and you would like to wait for that to finish, you can just call `Wait` instead.) If the operation then fails, `Result` throws an exception (as does `Wait`), although that is not as straightforward as you might expect, as I will discuss in [“Error Handling”](#).

WARNING

You should avoid using `Result` on an uncompleted task. In some scenarios, it risks deadlock, as does `Wait`. This is particularly common in desktop applications, because certain work needs to happen on particular threads, and if you block a thread by reading the `Result` of an incomplete task, you might prevent the task from completing. Even if you don't deadlock, blocking on `Result` can cause performance issues by hogging thread pool threads that might otherwise have been able to get on with useful work. And reading the `Result` of an uncompleted `ValueTask<T>` is not permitted.

In most cases, it is far better to use C#'s asynchronous language features to retrieve the result. These are the subject of the next chapter, but as a preview, [Example 16-13](#) shows how you could use this to get the result of the task that fetches a web page. (You'll need to apply the `async` keyword in front of the method declaration to be able to use the `await` keyword.)

Example 16-13. Getting a task's results with `await`

```
string pageContent = await webGetTask;
```

This may not look like an exciting improvement on simply writing `webGetTask.Result`, but as I'll show in [Chapter 17](#), this code is not quite what it seems—the C# compiler restructures this statement into a call-back-driven state machine that enables you to get the result without blocking the calling thread. (If the operation hasn't finished, the thread returns to the caller, and the remainder of the method runs later when the operation completes.)

But how are the asynchronous language features able to make this work—how can code discover when a task has completed? `Result` or `Wait` let you just sit and wait for that to happen, blocking the thread, but that rather defeats the purpose of using an asynchronous API in the first place. You will normally want to be notified when the task completes, and you can do this with a *continuation*.

Continuations

Tasks provide various overloads of a method called `ContinueWith`. This creates a new thread-based task that will execute when the task on which you called `ContinueWith` finishes (whether it does so successfully or with failure or cancellation). [Example 16-14](#) uses this on the task created in [Example 16-12](#).

Example 16-14. A continuation

```
webGetTask.ContinueWith(static t =>
{
    string webContent = t.Result;
    Console.WriteLine($"Web page length: {webContent.Length}");
});
```

A continuation task is always a thread-based task (regardless of whether its antecedent task was thread-based, I/O-based, or something else). The task gets created as soon as you call `ContinueWith` but does not become runnable until its antecedent task completes. (It starts out in the `WaitingForActivation` state.)

NOTE

A continuation is a task in its own right—`ContinueWith` returns either a `Task<T>` or `Task`, depending on whether the delegate you supply returns a result. You can set up a continuation for a continuation if you want to chain together a sequence of operations.

The method you provide for the continuation (such as the lambda in [Example 16-14](#)) receives the antecedent task as its argument, and I've used this to retrieve the result. I could also have used the `webGetTask` variable, which is in scope from the containing method, as it refers to the same task. However, by using the argument, the lambda in [Example 16-14](#) doesn't use any variables from its containing method, which enables the compiler to produce slightly more efficient code—it doesn't need to create an object to hold shared variables, and it can reuse the delegate instance

it creates because it doesn't have to create a context-specific one for each call. (I put the `static` method on the lambda to tell the compiler that this was my intention. It would still generate the more efficient code even without that keyword, but this way if I accidentally tried to capture a variable, I'd get a compiler error instead of the compiler silently generating less efficient code.) This means I could also easily separate this out into an ordinary noninline method, if I felt that would make the code easier to read.

You might be thinking that there's a possible problem in [Example 16-14](#): What if the download completes extremely quickly so that `webGetTask` has already completed before the code manages to attach the continuation? In fact, that doesn't matter—if you call `ContinueWith` on a task that has already completed, it will still run the continuation. It just schedules it immediately. You can attach as many continuations as you like. All the continuations you attach before the task completes will be scheduled for execution when it does complete. And any that you attach after the task has completed will be scheduled immediately.

By default, a continuation task will be scheduled for execution on the thread pool like any other task. However, there are some things you can do to change how it runs. Some overloads of `ContinueWith` take an argument of the `enum` type `TaskContinuationOptions`, which controls how (and whether) your task is scheduled. This includes all of the same options that are available with `TaskCreationOptions` but adds some others specific to continuations.

You can specify that the continuation should run only in certain circumstances. For example, the `OnlyOnRanToCompletion` flag will ensure that the continuation runs only if the antecedent task succeeds. There are similar `OnlyOnFaulted` and `OnlyOnCanceled` flags. Alternatively, you can specify `NotOnRanToCompletion`, which means that the continuation will run only if the task either faults or is canceled.

NOTE

You can create multiple continuations for a single task. So you could set up one to handle the success case and another one to handle failures.

You can also specify `ExecuteSynchronously`. This indicates that the continuation should not be scheduled as a separate work item. Normally, when a task completes, any continuations for that task will be scheduled for execution and will have to wait for the normal thread pool mechanisms to pick the work items out of the queue and execute them. (This won't take long if you use the default options. Unless you specify `PreferFairness`, the LIFO operation the thread pool uses for tasks means that the most recently scheduled items run first.) However, if your completion does only the tiniest amount of work, the overhead of scheduling it as a completely separate item may be overkill. So `ExecuteSynchronously` lets you piggyback the completion task on the same thread pool work item that ran the antecedent. The TPL will run this kind of continuation immediately after the antecedent finishes before returning the thread to the pool. You should use this option only if the continuation will run quickly.

The `LazyCancellation` option handles a tricky situation that can occur if you make tasks cancelable (as described in [“Cancellation”](#)) and you are using continuations. If you cancel a task, any continuations will, by default, become runnable instantly. If the task being canceled was itself set up as a continuation for another task that hadn't yet finished, and if it has a continuation of its own, as in [Example 16-15](#), this can have a mildly surprising effect.

Example 16-15. Cancellation and chained continuations

```
private static void ShowContinuations()
{
    Task op = Task.Run(DoSomething);
    var cs = new CancellationTokenSource();
    Task onDone = op.ContinueWith(
        _ => Console.WriteLine("Never runs"),
        cs.Token);
}
```

```

        Task andAnotherThing = onDone.ContinueWith(
            _ => Console.WriteLine("Continuation's continuation"));
        cs.Cancel();
    }

    static void DoSomething()
    {
        Thread.Sleep(1000);
        Console.WriteLine("Initial task finishing");
    }

```

This creates a task that will call `DoSomething`, followed by a cancelable continuation for that task (the `Task` in `onDone`), and then a final task (`andAnotherThing`) that is a continuation for the first continuation. This code cancels the first continuation almost immediately, which is almost certain to happen before the first task completes. The effect of this is that the final task runs before the first completes. The final `andAnotherThing` task becomes runnable when `onDone` completes, even if that completion was due to `onDone` being canceled. Since there was a chain here—`andAnotherThing` is a continuation for `onDone`, which is a continuation for `op`—it is a bit odd that `andAnotherThing` ends up running before `op` has finished. `LazyCancellation` changes the behavior so that the first continuation will not be deemed to have completed until its antecedent completes, meaning that the final continuation will run only after the first task has finished.

There's another mechanism for controlling how tasks execute: you can specify a scheduler.

Schedulers

All thread-based tasks are executed by a `TaskScheduler`. By default, you'll get the TPL-supplied scheduler that runs work items via the thread pool. However, there are other kinds of schedulers, and you can even write your own.

The most common reason for selecting a nondefault scheduler is to handle thread affinity requirements. The `TaskScheduler` class's static

`FromCurrentSynchronizationContext` method returns a scheduler based on the current synchronization context for whichever thread you call the method from. This scheduler will execute all work via that synchronization context.

So, if you call `FromCurrentSynchronizationContext` from a UI thread, the resulting scheduler can be used to run tasks that can safely update the UI. You would typically use this for a continuation—you can run some task-based asynchronous work and then hook up a continuation that updates the UI when that work is complete. [Example 16-16](#) shows this technique in use in the codebehind file for a window in a WPF application.

Example 16-16. Scheduling a continuation on the UI thread

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private static readonly HttpClient w = new();
    private readonly TaskScheduler _uiScheduler =
        TaskScheduler.FromCurrentSynchronizationContext();

    private void FetchButtonClicked(object sender, RoutedEventArgs e)
    {
        Task<string> webGetTask = w.GetStringAsync("https://endjin.com/");

        webGetTask.ContinueWith(t =>
        {
            string webContent = t.Result;
            outputTextBox.Text = webContent;
        },
        _uiScheduler);
    }
}
```

This uses a field initializer to obtain the scheduler—the constructor for a UI element runs on the UI thread, so this will get a scheduler for the synchronization context for the UI thread. A click handler then downloads a web page using the `HttpClient` class's `GetStringAsync`. This runs asynchronously, so it won't block the UI thread, meaning that the application will remain responsive while the download is in progress. The method sets up a continuation for the task using an overload of `ContinueWith` that takes a `TaskScheduler`. This ensures that when the task that gets the content completes, the lambda passed to `ContinueWith` runs on the UI thread, so it's safe for it to access UI elements.

TIP

While this works perfectly well, the `await` keyword described in the next chapter provides a more straightforward solution to this particular problem.

The runtime libraries provide three built-in kinds of schedulers. There's the default one that uses the thread pool, and the one I just showed that uses a synchronization context. The third is provided by a class called `ConcurrentExclusiveSchedulerPair`, and as the name suggests, this provides two schedulers, which it makes available through properties. The `ConcurrentScheduler` property returns a scheduler that will run tasks concurrently much like the default scheduler. The `ExclusiveScheduler` property returns a scheduler that can be used to run tasks one at a time, and it will temporarily suspend the other scheduler while it does so. (This is reminiscent of `ReaderWriterLockSlim`—it allows exclusivity when required but concurrency the rest of the time.)

Error Handling

A `Task` object indicates when its work has failed by entering the `Faulted` state. There will always be at least one exception associated with failure, but the TPL allows composite tasks—tasks that contain a number of subtasks. This makes it possible for multiple failures to occur, and the root task will report them all. `Task` defines an `Exception` property, and its type is `AggregateException`. You may recall from [Chapter 8](#)

that as well as inheriting the `InnerException` property from the base `Exception` type, `AggregateException` defines an `InnerExceptions` property that returns a collection of exceptions. This is where you will find the complete set of exceptions that caused the task to fault. (If the task was not a composite task, there will usually be just one.)

If you attempt to get the `Result` property or call `Wait` on a faulted task, it will throw the same `AggregateException` as it would return from the `Exception` property. A faulted task remembers whether you have used at least one of these members, and if you have not yet done so, it considers the exception to be *unobserved*. The TPL uses finalization to track faulted tasks with unobserved exceptions, and if you allow such a task to become unreachable, the `TaskScheduler` will raise its static `UnobservedTaskException` event. This gives you one last chance to do something about the exception, after which it will be lost.

Custom Threadless Tasks

Many I/O-based APIs return threadless tasks. You can do the same if you want. The `TaskCompletionSource<T>` class provides a way to create a `Task<T>` that does not have an associated method to run on the thread pool and instead completes when you tell it to. There's also a nongeneric `TaskCompletionSource` for creating a nongeneric `Task`. (.NET Framework doesn't have the nongeneric one, but since `Task<T>` derives from `Task`, you can just use `TaskCompletionSource<object?>` instead.)

Suppose you're using a class that does not provide a task-based API, and you'd like to add a task-based wrapper. The runtime libraries provide an `SmtpClient` class for sending emails, and it supports an older event-based asynchronous pattern but not the task-based one. [Example 16-17](#) uses that API in conjunction with `TaskCompletionSource<object?>` to provide a task-based wrapper.

Example 16-17. Using `TaskCompletionSource<T>`

```
public static class SmtpAsyncExtensions
{
```



```

public static Task SendTaskAsync(this SmtplibClient mailClient, string from,
    string recipients, string subject, string body)
{
    var tcs = new TaskCompletionSource<object?>();

    void CompletionHandler(object s, AsyncCompletedEventArgs e)
    {
        // Check this is the notification for our SendAsync.
        if (!object.ReferenceEquals(e.UserState, tcs)) { return; }
        mailClient.SendCompleted -= CompletionHandler;
        if (e.Canceled)
        {
            tcs.SetCanceled();
        }
        else if (e.Error != null)
        {
            tcs.SetException(e.Error);
        }
        else
        {
            tcs.SetResult(null);
        }
    };

    mailClient.SendCompleted += CompletionHandler;
    mailClient.SendAsync(from, recipients, subject, body, tcs);

    return tcs.Task;
}
}

```

The `SmtplibClient` notifies us that the operation is complete by raising an event. The handler for this event first checks that the event corresponds to our call to `SendAsync` and not some other operation that may have already been in progress. It then detaches itself (so that it doesn't run a second time if something uses that same `SmtplibClient` for further work). Then it detects whether the operation succeeded, was canceled, or failed, and calls the `SetResult`, `SetCanceled`, or `SetException` method, respectively, on the `TaskCompletionSource<object?>`. This will cause the task to transition into the relevant state and will also take care of running any continuations attached to that task. The completion source makes the

threadless `Task` object it creates available through its `Task` property, which this method returns.

Parent/Child Relationships

If a thread-based task's method creates a new thread-based task, then by default, there will be no particular relationship between those tasks. However, one of the `TaskCreationOptions` flags is `AttachedToParent`, and if you set this, the newly created task will be a child of the task currently executing. The significance of this is that the parent task won't report completion until all its children have completed. (Its own method also needs to complete, of course.) If any children fault, the parent task will fault, and it will include all the children's exceptions in its own `AggregateException`.

You can also specify the `AttachedToParent` flag for a continuation. Be aware that this does not make it a child of its antecedent task. It will be a child of whichever task was running when `ContinueWith` was called to create the continuation.

NOTE

Threadless tasks (e.g., most tasks representing I/O) often cannot be made children of another task. If you create one yourself with a `TaskCompletionSource<T>`, you can do it because that class has a constructor overload that accepts a `TaskCreationOptions`. However, the majority of .NET APIs that return tasks do not provide a way to request that the task be a child.

Parent/child relationships are not the only way of creating a task whose outcome is based on multiple other items.

Composite Tasks

The `Task` class has static `WhenAll` and `WhenAny` methods. Each of these has overloads that accept either a collection of `Task` objects or a collection of `Task<T>` objects as the only argument. The `WhenAll` method re-

turns either a `Task` or a `Task<T[]>` that completes only when all of the tasks provided in the argument have completed (and in the latter case, the composite task produces an array containing each of the individual tasks' results). The `WhenAny` method returns a `Task<Task>` or `Task<Task<T>>` that completes as soon as the first task completes, providing that task as the result.

As with a parent task, if any of the tasks that make up a task produced with `WhenAll` fail, the exceptions from all of the failed tasks will be available in the composite task's `AggregateException`. (`WhenAny` does not report errors. It completes as soon as the first task completes, and you must inspect that to discover if it failed.)

You can attach a continuation to these tasks, but there's a slightly more direct route. Instead of creating a composite task with `WhenAll` or `WhenAny` and then calling `ContinueWith` on the result, you can just call the `ContinueWhenAll` or `ContinueWhenAny` method of a task factory. Again, these take a collection of `Task` or `Task<T>`, but they also take a method to invoke as the continuation.

Other Asynchronous Patterns

Although the TPL provides the preferred mechanism for exposing asynchronous APIs, .NET had been around for almost a decade before it was added, so you will come across older approaches. The longest established form is the Asynchronous Programming Model (APM). This was introduced in .NET 1.0, so it is widely implemented, but its use is now discouraged. With this pattern, methods come in pairs: one to start the work and a second to collect the results when it is complete. [Example 16-18](#) shows just such a pair from the `Stream` class in the `System.IO` namespace, and it also shows the corresponding synchronous method. (Code written today should use a task-based `WriteAsync` instead.)

Example 16-18. An APM pair and the corresponding synchronous method

```
public virtual IAsyncResult BeginWrite(byte[] buffer, int offset, int count,
    AsyncCallback callback, object state)...
public virtual void EndWrite(IAsyncResult asyncResult)...

public abstract void Write(byte[] buffer, int offset, int count)...
```

Notice that the first three arguments of the `BeginWrite` method are identical to those of the `Write` method. In the APM, the `BeginXxx` method takes all of the inputs (i.e., any normal arguments and any `ref` arguments but not `out` arguments, should any be present). The `EndXxx` method provides any outputs, which means the return value, any `ref` arguments (because those can pass information either in or out), and any `out` arguments.

The `BeginXxx` method also takes two additional arguments: a delegate of type `AsyncCallback`, which will be invoked when the operation completes, and an argument of type `object` that accepts any object you would like to associate with the operation (or `null` if you have no use for this). This method also returns an `IAsyncResult`, which represents the asynchronous operation.

When your completion callback gets invoked, you can call the `EndXxx` method, passing in the same `IAsyncResult` object returned by the `BeginXxx` method, and this will provide the return value if there is one. If the operation failed, the `EndXxx` method will throw an exception.

You can wrap APIs that use the APM with a `Task`. The `TaskFactory` objects provided by `Task` and `Task<T>` provide `FromAsync` methods to which you can pass a pair of delegates for the `BeginXxx` and `EndXxx` methods, and you also pass any arguments that the `BeginXxx` method requires. This will return a `Task` or `Task<T>` that represents the operation.

Another common older pattern is the Event-based Asynchronous Pattern (EAP). You've seen an example in this chapter—it's what the `SmtpClient` uses. With this pattern, a class provides a method that starts the opera-

tion and a corresponding event that it raises when the operation completes. The method and event usually have related names, such as `SendAsync` and `SendCompleted`. An important feature of this pattern is that the method captures the synchronization context and uses that to raise the event, meaning that if you use an object that supports this pattern in UI code, it effectively presents a single-threaded asynchronous model. This makes it much easier to use than the APM, because you don't need to write any extra code to get back onto the UI thread when asynchronous work completes.

There's no automated mechanism for wrapping the EAP in a task, but as I showed in [Example 16-17](#), it's not particularly hard to do.

There's one more common pattern used in asynchronous code: the *awaitable* pattern supported by the C# asynchronous language features (the `async` and `await` keywords). As I showed in [Example 16-13](#), you can consume a TPL task directly with these features, but the language does not recognize `Task` directly, and it's possible to await things other than tasks. You can use the `await` keyword with anything that implements a particular pattern. I will show this in [Chapter 17](#).

Cancellation

.NET defines a standard mechanism for canceling slow operations. Cancelable operations take an argument of the type `CancellationToken`, and if you set this into a canceled state, the operation will stop early if possible instead of running to completion.

Note that the `CancellationToken` type itself does not offer any methods to initiate cancellation—the API is designed so that you can tell operations when you want them to be canceled without giving them power to cancel whatever other operations you have associated with the same `CancellationToken`. The act of cancellation is managed through a separate object, `CancellationTokenSource`. As the name suggests, you can use this to get hold of any number of `CancellationToken` instances. If

you call the `CancellationTokenSource` object's `Cancel` method, that sets all of the associated `CancellationToken` instances into a canceled state.

Some of the synchronization mechanisms I described earlier can be passed a `CancellationToken`. (`Monitor` does not support cancellation, but many newer APIs do.) It's also common for task-based APIs to take a cancellation token, and the TPL itself also offers overloads of the `StartNew` and `ContinueWith` methods that take them. If the task has already started to run, there's nothing the TPL can do to cancel it, but if you cancel a task before it begins to run, the TPL will take it out of the scheduled task queue for you. If you want to be able to cancel your task after it starts running, you'll need to write code in the body of your task that inspects the `CancellationToken` and abandons the work if its `IsCancellationRequested` property is `true`.

Cancellation support is not ubiquitous, because it's not always possible. Some operations simply cannot be canceled. For example, once a message has been sent out over the network, you can't unsend it. Some operations allow work to be canceled up until some point of no return has been reached. (If a message is queued up to be sent but hasn't actually been sent, then it might not be too late to cancel, for example.) This means that even when cancellation is offered, it might not do anything. So, when you use cancellation, you need to be prepared for it not to work.

Parallelism

The runtime libraries include some classes that can work with collections of data concurrently on multiple threads. There are three ways to do this: the `Parallel` class, Parallel LINQ, and TPL Dataflow.

The Parallel Class

The `Parallel` class offers five static methods: `For`, `ForEachAsync`, `ForEach`, `ForEachAsync`, and `Invoke`. The last of those takes an array of delegates and executes all of them, potentially in parallel. (Whether it decides to use parallelism depends on various factors such as the number of hard-

ware threads the computer has, how heavily loaded the system is, and how many items you want it to process.) The `For` and `ForEach` methods mimic the C# loop constructs of the same names, but they will also potentially execute iterations in parallel. `ForAsync` (new in .NET 8.0) and `ForEachAsync` also mimic these loop types, but they provide better support for asynchronous operation. Both accept a delegate that returns a task, enabling each iteration to perform asynchronous operations (equivalent to using `await` in the body of a `foreach` loop). `ForEachAsync` can work with `IAsyncEnumerable<T>` (like `await foreach`).

[Example 16-19](#) illustrates the use of `Parallel.For` in code that performs a convolution of two sets of samples. This is a highly repetitive operation commonly used in signal processing. (In practice, a fast Fourier transform offers a more efficient way to perform this work unless the convolution kernel is small, but the complexity of that code would have obscured the main subject here, the `Parallel` class.) It produces one output sample for each input sample. Each output sample is produced by calculating the sum of a series of pairs of values from the two inputs, multiplied together. For large data sets, this can be time consuming, so it is the sort of work you might want to speed up by spreading it across multiple processors. Each individual output sample's value can be calculated independently of all the others, so it is a good candidate for parallelization.

Example 16-19. Parallel convolution

```
static float[] ParallelConvolution(float[] input, float[] kernel)
{
    float[] output = new float[input.Length];
    Parallel.For(0, input.Length, i =>
    {
        float total = 0;
        for (int k = 0; k < Math.Min(kernel.Length, i + 1); ++k)
        {
            total += input[i - k] * kernel[k];
        }
        output[i] = total;
    });
}
```



```
    return output;  
}
```

The basic structure of this code is very similar to a pair of nested `for` loops. I've simply replaced the outer `for` loop with a call to `Parallel.For`. (I've not attempted to parallelize the inner loop—if you make each individual step trivial, `Parallel.For` will spend more of its time in housekeeping work than it does running your code.)

The first argument, `0`, sets the initial value of the loop counter, and the second sets the upper limit. The final argument is a delegate that will be invoked once for each value of the loop counter, and the calls will occur concurrently if the `Parallel` class's heuristics tell it that this is likely to produce a speedup as a result of the work running in parallel. Running this method with large data sets on a multicore machine causes all of the available hardware threads to be used to full capacity.

It may be possible to get better performance by partitioning the work in more cache-friendly ways—naive parallelization can give the impression of high performance by maxing out all your CPU cores while delivering suboptimal throughput. However, there is a trade-off between complexity and performance, and the simplicity of the `Parallel` class can often provide worthwhile wins for relatively little effort.

Parallel LINQ

Parallel LINQ is a LINQ provider that works with in-memory information, much like LINQ to Objects. The `System.Linq` namespace makes this available as an extension method called `AsParallel` defined for any `IEnumerable<T>` (by the `ParallelEnumerable` class). This returns a `ParallelQuery<T>`, which supports the usual LINQ operators.

Any LINQ query built this way provides a `ForAll` method, which takes a delegate. When you call this, it invokes the delegate for all of the items that the query produces, and it will do so in parallel on multiple threads where possible.

TPL Dataflow

TPL Dataflow is a runtime library feature that lets you construct a graph of objects that perform some kind of processing on information that flows through them. You can tell the TPL which of these nodes needs to process information sequentially and which are happy to work on multiple blocks of data simultaneously. You push data into the graph, and the TPL will then manage the process of providing each node with blocks to process, and it will attempt to optimize the level of parallelism to match the resources available on your computer.

The dataflow API is in the `System.Threading.Tasks.Dataflow` namespace. (It's built into .NET; on .NET Framework you'll need to add a reference to a NuGet package, also called `System.Threading.Tasks.Dataflow`.) It is large and complex and could have a whole chapter to itself. Sadly, this makes it beyond the scope of this book. I mention it because it's worth being aware of for certain kinds of work.

Summary

Threads provide the ability to execute multiple pieces of code simultaneously. On a computer with multiple CPU execution units (i.e., multiple hardware threads), you can exploit this potential for parallelism by using multiple software threads. You can create new software threads explicitly with the `Thread` class, or you can use either the thread pool or a parallelization mechanism, such as the `Parallel` class or Parallel LINQ, to determine automatically how many threads to use to run the work your application supplies. If multiple threads need to use and modify shared data structures, you will need to use the synchronization mechanisms offered by .NET to ensure that the threads can coordinate their work correctly.

Threads can also provide a way to execute multiple concurrent operations that do not need the CPU the whole time (e.g., waiting for a response from an external service), but it is often more efficient to perform such work with asynchronous APIs (where available). The Task Parallel

Library (TPL) provides abstractions that are useful for both kinds of concurrency. It can manage multiple work items in the thread pool, with support for combining multiple operations and handling potentially complex error scenarios, and its `Task` abstraction can also represent inherently asynchronous operations. The next chapter describes C# language features that greatly simplify working with tasks.

1 I'm using the word *state* here broadly. I just mean information stored in variables and objects.

2 At the time of this writing, the documentation does not offer read-only thread safety guarantees for `HashSet<T>` and `SortedSet<T>`. Nonetheless, I have been assured by Microsoft that these also support concurrent reads.